

EPICA 1 - Arquitectura e Infraestructura

- **Objetivo de la Épica:** Establecer el entorno fundacional estandarizado que garantice la integración continua (CI), reproducibilidad del entorno de desarrollo y la persistencia de datos espaciales. Al finalizar esta épica, ambos desarrolladores podrán ejecutar el stack completo con un solo comando.
- **Métrica de Éxito:** Tiempo de configuración de un nuevo entorno (Onboarding Time) < 15 minutos.

HU1: Configuración del entorno y versionamiento

Descripción: Como desarrollador de datos, necesito un repositorio configurado con Docker y convenciones de control de versiones para evitar el problema de "en mi máquina sí funciona".

- **Criterios de Aceptación (Gherkin):**
 - Feature: Entorno de Desarrollo Reproducible

Scenario: Levantar el entorno base mediante contenedores

Given que el repositorio de GitHub ha sido clonado localmente

And el archivo 'docker-compose.yml' contiene los servicios base (PostgreSQL, FastAPI)

When el desarrollador ejecuta el comando "docker-compose up -d"

Then los contenedores deben levantar sin errores

And el servicio de base de datos debe estar escuchando en el puerto 5432

And la API base debe responder con estado 200 OK en "localhost:8000/health"

Scenario: validación de las credenciales que NO están hardcodeadas en el repositorio

Given el archivo docker-compose.yml,

When se lee su contenido,

Then no debe contener ningún string que coincida con el patrón de contraseña en texto plano

HU2: Documentación de Arquitectura de Datos

Descripción: Como arquitecto del sistema, necesito un documento formal de diseño (Architecture Decision Record) para establecer el flujo de los pipelines ETL y la comunicación Frontend-Backend.

- **Criterios de Aceptación (Gherkin):**

Gherkin

Feature: Diseño Arquitectónico (ADR)

Scenario: Definición del flujo de datos

Given la necesidad de procesar datos del SIATA y DANE

When se redacta el documento ADR de Arquitectura

Then debe incluir un diagrama C4 o de flujo de datos

And debe justificar la elección de FastAPI sobre Flask y PostgreSQL sobre MongoDB

And debe estar almacenado en la carpeta "/docs/architecture" del repositorio

HU3: Despliegue de Base de Datos Geoespacial (PostGIS)

Descripción: Como ingeniero de datos, necesito una base de datos PostgreSQL con la extensión PostGIS habilitada para poder almacenar y consultar las coordenadas y polígonos de los municipios de Colombia.

- **Criterios de Aceptación (Gherkin):**

Gherkin

Feature: Almacenamiento Geoespacial

Scenario: Inicialización de la base de datos con capacidades espaciales

Given el contenedor de PostgreSQL está en ejecución

When se ejecuta el script de inicialización SQL (init.sql)

Then la extensión "postgis" debe ser instalada en la base de datos "vitalrisk_db"

And se debe crear la tabla base "dim_municipios" con una columna de tipo "GEOMETRY"

Scenario: todas las tablas del esquema están creadas, no solo la dimensión de municipios.

Given el script init.sql ejecutado,

When se consulta el catálogo de tablas,

Then deben existir las tablas fact_calidad_aire, fact_eventos_ira, fact_riesgo_territorial y alertas_territoriales

HU4: Tablero Jira y Workflow Ágil

- **Descripción:** Como Scrum Master del equipo, necesito un tablero de Jira configurado con el workflow adecuado (To Do, In Progress, Code Review, Done) para dar seguimiento a la velocidad del equipo.
- **Criterios de Aceptación:** *(No requiere Gherkin al no ser código, sino configuración de proceso).*
 - **Definition of Done (DoD) configurado:** Todo ticket en 'Done' debe tener código en la rama main, validado, y sin credenciales hardcodeadas.

EPICA 2 - Pipelines de Datos (ETL) e Integración Territorial

- **Objetivo de la Épica:** Construir flujos automatizados e idempotentes que extraigan los datos abiertos, los limpien, los crucen espacial y temporalmente, y los carguen en la base de datos PostGIS.
- **Métrica de Éxito (Calidad de Datos - DQS):** Completitud territorial > 95% (que casi todos los municipios objetivo tengan datos cruzados) y 0% de duplicados en la tabla final.

HU5: Extracción (Ingesta) de Datos Abiertos

Descripción: Como ingeniero de datos, necesito scripts en Python que descarguen los datasets oficiales (CSV o API Socrata) y los almacenen en su estado crudo (*raw*) localmente para tener un snapshot estático y no depender de internet en cada prueba.

- **Criterios de Aceptación (Gherkin):**

Gherkin

Feature: Ingesta de datos crudos (Raw Data)

Scenario: Descarga de datos del INS y SIATA

Given que las URLs o endpoints de las APIs están definidos en variables de entorno

When se ejecuta el script "extract_data.py"

Then los archivos crudos se guardan en el directorio "/data/raw/"

And los nombres de archivo incluyen un timestamp (ej. "ins_ira_20260524.csv")

Scenario: Validación de columnas clave post-descarga

Given el archivo crudo descargado en "/data/raw/"

When el script de validación analiza el esquema del archivo

Then debe confirmar la presencia de al menos una columna de código territorial (DIVIPOLA)

And debe confirmar la presencia de al menos una columna temporal (fecha o semana)

And si alguna columna clave falta, genera un log de advertencia en lugar de fallar silenciosamente

HU6: Limpieza y Normalización (Data Cleaning)

- **Descripción:** Como científico de datos, necesito estandarizar los formatos de fechas, nombres de columnas y manejar valores nulos para poder procesarlos matemáticamente.
- **Criterios de Aceptación (Gherkin):**

Gherkin

Feature: Limpieza de datasets

Scenario: Normalización de formatos

Given los archivos en "/data/raw/"

When el pipeline ejecuta el módulo de limpieza usando Pandas

Then todas las fechas se transforman al formato estándar ISO 8601 (YYYY-MM-DD)

And los nombres de las columnas se convierten a "snake_case" (ej. "Casos_Totales" -> "casos_totales")

And los valores nulos numéricos son imputados o marcados explícitamente

And el resultado se guarda en "/data/processed/clean_*.csv"

Scenario: Marcado del período de pandemia

Given los registros con fecha entre 2020-03-01 y 2021-12-31

When el pipeline de limpieza procesa las fechas

Then se crea la columna "periodo_pandemia" con valor True para ese rango

And el resto de registros tiene periodo_pandemia = False

And este campo se documenta en el Data Dictionary

Scenario: Filtrado del rango temporal del proyecto

Given los datasets con registros históricos

When el pipeline aplica el filtro temporal

Then solo se conservan registros desde 2018-01-01 hasta 2023-12-31

And los registros fuera de ese rango se loggean y descartan, no se pierden silenciosamente

ACLARO, EL EJEMPLO ANTERIOR, ES RESPECTO A UN TIPO DE DATOS, PERO HAY QUE REVISAR TODOS LOS DATOS

HU7: Integración Territorial y Creación de la Tabla Maestra (Feature Store)

- **Descripción:** Como analista, necesito un único dataset (Tabla Maestra) que cruce la información epidemiológica y la ambiental utilizando el Código DANE del municipio y la fecha (mes/semana epidemiológica).

-- tasa = (casos_ira_total / poblacion_anio) * 100000

-- donde poblacion_anio viene del JOIN con dim_poblacion_anual

-- usando (codigo_dane, anio) como llave

- **Criterios de Aceptación (Gherkin):**

Gherkin

Feature: Cruce espacial y temporal (Join)

Scenario: Construcción de la Tabla Analítica Maestra

Given los datasets limpios de salud y ambiente

When se ejecuta el proceso de cruce (Merge)

Then el cruce se realiza utilizando "codigo_dane" y "semana_epidemiologica" como llaves compuestas

And la complejidad computacional del cruce es optimizada evitando duplicidad cartesiana

And la tabla resultante contiene columnas de exposición (PM2.5) y respuesta (Casos IRA)

Scenario: Manejo de municipios sin estación de monitoreo

Given municipios del Valle de Aburrá sin estación SIATA/IDEAM propia

When el cruce intenta asignar PM2.5 a esos municipios

Then se usa la medición de la estación más cercana (interpolación por proximidad)

And la columna "fuente_pm25" registra si el valor es directo o interpolado

And al menos el 80% de los municipios del Valle de Aburrá tienen PM2.5 asignado

Scenario: Completitud de la Tabla Maestra

Given el cruce ejecutado sobre datos 2018-2023

When se evalúa la cobertura del resultado

Then la tabla debe tener al menos 200 registros (10 municipios x 5 años x 52 semanas aprox)

And la columna "casos_ira_total" no debe tener más del 15% de nulos

And la columna "pm25_semanal_avg" no debe tener más del 30% de nulos

HU8: Carga en Base de Datos Geoespacial (PostGIS)

- **Descripción:** Como desarrollador Backend, necesito que la tabla maestra estructurada sea cargada en PostgreSQL/PostGIS para que la API (FastAPI) pueda consultar los datos eficientemente mediante queries SQL.
- **Criterios de Aceptación (Gherkin):**

Gherkin

Feature: Carga de datos en PostGIS (Load)

Scenario: Inserción idempotente de la Tabla Maestra

Given la Tabla Analítica Maestra generada en formato DataFrame o CSV

When se ejecuta el script "load_to_db.py" a través de SQLAlchemy/GeoAlchemy

Then los datos se insertan en la tabla "fact_riesgo_territorial"

And si un registro ya existe (mismo municipio y fecha), se actualiza (Upsert) sin generar error ni duplicados

Scenario: Validación de integridad referencial pre-carga

Given el DataFrame de la Tabla Maestra

When el script verifica los códigos DANE antes de la carga

Then todos los codigos_dane del DataFrame deben existir en dim_municipios

And si hay codigos_dane no encontrados, se loglean como "municipios_sin_dimension"

And la carga continúa solo con los registros con codigos válidos

EPICA 3 - Análisis Exploratorio e Índice Preventivo Territorial:

- **Objetivo de la Épica:** Desarrollar, validar y persistir la métrica determinista de riesgo (IPT) mediante análisis estadístico exploratorio y un algoritmo de cálculo vectorizado, garantizando su acotamiento matemático mediante pruebas de caja negra.

- **Métrica de Éxito:** Cobertura de pruebas unitarias (Unit Test Coverage) del 100% sobre el módulo de cálculo del IPT, verificando que el 100% de los municipios procesados posean un valor válido $\$IPT \in [0, 100]\$$.

HU9: Análisis de Correlación y Perfilamiento

- **Descripción:** Como científico de datos, necesito perfilar estadísticamente la base de datos unificada para extraer los pesos empíricos ($\$w_i\$$) que determinarán la influencia de cada variable ambiental sobre las enfermedades respiratorias.
- **Criterios de Aceptación (Gherkin):**

Gherkin

Feature: Perfilamiento estadístico (EDA)

Scenario: Selección de features predictivos

Given la tabla maestra "fact_riesgo_territorial" poblada

When se ejecuta el script de correlación de Pearson

Then el sistema genera una matriz de correlación

And las variables con una correlación p-value < 0.05 respecto a "casos_ira" son marcadas como "activas"

And las variables redundantes (colinealidad > 0.90 entre sí) son excluidas

Scenario: Selección robusta de features (no-paramétrico)

Given variables con distribución no-normal confirmada por test Shapiro-Wilk

When se aplica correlación de Spearman en lugar de Pearson

Then las variables con $\rho > 0.3$ y p-value < 0.05 también son marcadas como "activas"

And el reporte indica qué método se usó por variable

HU10: Tratamiento de Anomalías (Winsorización)

- **Descripción:** Como científico de datos, necesito analizar las distribuciones estadísticas de las variables unificadas para aplicar tratamientos de imputación o

truncamiento de valores extremos que puedan sesgar el comportamiento del modelo predictivo.

- **Criterios de Aceptación:**

Gherkin

Feature: Ingeniería de Características y Tratamiento de Outliers

Feature: Manejo de Outliers mediante Winsorización

Scenario: Acotamiento de percentiles extremos

Given un DataFrame con variables continuas que presentan asimetría severa

When se aplica la función de winsorización a los datos

Then los valores por debajo del percentil 1% y por encima del 99% son reemplazados por los valores de dichos límites

And la varianza de la variable se estabiliza sin perder volumen de datos

Ejemplo:

Scenario: Winsorización de variables ambientales

Given el DataFrame "fact_riesgo_territorial" cargado en memoria

When la concentración de PM2.5 excede el percentil 99 o no llega al percentil 1

Then el valor es acotado rígidamente al umbral del percentil por debajo de 1 y encima del percentil 99 para evitar sesgos en la varianza

And la distribución resultante preserva la integridad estructural de la serie de tiempo

-

HU11: Cálculo del IPT (Índice Preventivo Territorial)

Descripción: Como ingeniero de software, necesito implementar el algoritmo que calcule la función matemática del IPT de forma vectorizada (usando NumPy/Pandas para lograr una eficiencia $O(1)$ en operaciones matriciales), garantizando que el

resultado final se encuentre estrictamente acotado entre 0 y 100 para la correcta ingesta del modelo de Machine Learning.

Estrategia de Verificación y Pruebas (QA): Para validar esta funcionalidad, implementaremos una suite de pruebas unitarias automatizadas (pytest) basada en dos técnicas de caja negra:

- 1. **Partición de Equivalencia:** El dominio se divide en clases con comportamiento similar que definirán los niveles de riesgo: Bajo (0-33), Medio (34-66) y Alto (67-100).
- 2. **Análisis del Valor Límite:** Dado que la mayoría de los errores ocurren en las fronteras, comprobaremos los límites válidos e inválidos. Se inyectarán matrices que produzcan resultados crudos negativos (<0) o superiores a 100 (>100), verificando que la función limitadora (clipping) del algoritmo reasigne correctamente el valor a los umbrales de 0 o 100 respectivamente.

Feature: Cálculo vectorizado del IPT Scenario Outline:

Validación de umbrales y clases de equivalencia del índice

Given las variables estandarizadas para el municipio "i" en el tiempo "t"

When el motor matemático realiza el cálculo lineal y obtiene un <valor_crudo>

Then el sistema aplica la función limitadora (clipping)

And el <ipt_final> obedece la invariante $0 \leq IPT \leq 100$

And el sistema clasifica el riesgo en la <categoria> correspondiente

Ejemplo:

valor_crudo	ipt_final	categoria	justificacion_prueba
-5.0	0.0	Bajo	Limite no valido inferior (<0)
0.0	0.0	Bajo	Limite valido inferior (0)
15.5	15.5	Bajo	Particion de equivalencia 1
33.0	33.0	Bajo	Frontera interna inferior

34.0	34.0	Medio	Frontera interna superior
50.0	50.0	Medio	Particion de equivalencia 2
67.0	67.0	Alto	Frontera interna superior
100.0	100.0	Alto	Limite valido superior (100)
105.5	100.0	Alto	Limite no valido superior (>100)

Scenario: Eficiencia vectorizada del cálculo

Given un DataFrame con 10,000 registros de municipios y semanas

When se ejecuta calcular_ipt_vectorizado sobre el DataFrame completo

Then el tiempo de ejecución es menor a 1 segundo

And el resultado es idéntico al producido por calcular_ipt registro por registro

EPICA 4 - Machine Learning y Alertas Preventivas Territoriales

- **Objetivo de la Épica:** Entrenar, evaluar y desplegar un modelo de ensamble (XGBoost/LightGBM) que identifique patrones no lineales entre el clima, la contaminación y los casos respiratorios, generando alertas tempranas basadas en umbrales estadísticos.
- **Métrica de Éxito:** Un modelo entrenado con un $R^2 > 0.65$ (o un baseline razonable para datos ruidosos de salud) y un reporte de las top 5 variables más importantes.

HU12: Ingeniería de Características y Partición Temporal

- **Descripción:** Como científico de datos, necesito preparar el dataset maestro creando rezagos temporales (*lags*, ej. PM2.5 de la semana anterior) y dividiendo los datos cronológicamente para evitar fugas de información.
- **Criterios de Aceptación (Gherkin):**

Gherkin

Feature: Preparación del Dataset para ML

Scenario: Creación de variables rezagadas y split temporal

Given la tabla "fact_riesgo_territorial" validada

When se ejecuta el pipeline de Feature Engineering

Then se generan variables de rezago (lag_1_semana, lag_2_semanas) para PM2.5 y clima

And el dataset se divide en Train (hasta diciembre 2021), validacion(enero 2022, hasta diciembre 2022) Test (enero 2023 en adelante)

And se verifica que no exista superposición de fechas entre los conjuntos de entrenamiento y prueba

Scenario: Verificación de ausencia de data leakage temporal

Given el dataset dividido en Train, Validación y Test

When se verifica la separación temporal

Then la fecha máxima de Train es estrictamente menor a la fecha mínima de Validación

And la fecha máxima de Validación es estrictamente menor a la fecha mínima de Test

And ninguna variable del conjunto de Test fue usada para calcular estadísticas de imputación

HU13: Entrenamiento del Modelo Predictivo

- **Descripción:** Como científico de datos, necesito entrenar un modelo basado en árboles (Arboles de decision y metodos ensemble: Bagging, Boosting y Random Forest) priorizando la interpretabilidad de los resultados sobre la complejidad, para estimar la tendencia de casos IRA.

- **Criterios de Aceptación (Gherkin):**

Gherkin

Feature: Entrenamiento de modelo interpretable

Scenario: Ajuste del ensamble de árboles y extracción de importancia

Given los conjuntos de Train y Test preparados

When se entrena el regresor XGBoost con hiperparámetros base (max_depth=4, n_estimators=100)

Then el modelo converge sin errores de memoria

And el sistema extrae un arreglo con la "Feature Importance" (pesos relativos de cada variable)

And el artefacto del modelo se guarda en formato ".pkl" o ".joblib"

Scenario: Verificación de ausencia de data leakage en features

Given el modelo entrenado con variables de entrada

When se inspecciona el listado de features usadas

Then ninguna feature de entrada corresponde a la misma semana t de la variable objetivo

And todas las features de PM2.5 y casos IRA son versiones rezagadas (lag ≥ 1 semana)

HU14: Evaluación de Desempeño y Métricas

- **Descripción:** Como ingeniero de software, necesito un script automatizado que calcule el RMSE y MAE del modelo sobre el conjunto de Test para validar si sus predicciones superan un modelo estadístico simple (baseline).

- **Criterios de Aceptación (Gherkin):**

Gherkin

Feature: Evaluación de predicciones

Scenario: Cálculo de métricas de error

Given el modelo XGBoost entrenado (archivo .pkl)

And el conjunto de Test

When el modelo realiza inferencias (predicciones) sobre las variables independientes del Test

Then el sistema calcula la métrica RMSE mediante la fórmula matemática estándar

And calcula el MAE (Error Absoluto Medio)

And genera un reporte consolidado en "/docs/models/metrics_report.json"

Scenario: Comparación contra modelo baseline (naive)

Given el modelo XGBoost evaluado("U otro") y el conjunto de Test

When se calcula el RMSE de un predictor naive (media histórica de esa semana epidemiológica)

Then el RMSE del XGBoost debe ser al menos 15% menor que el RMSE del predictor naive

And este resultado se documenta en el reporte de métricas como justificación de valor

HU15: Motor Lógico de Alertas Preventivas

- **Descripción:** Como analista de salud, necesito que el sistema genere una bandera o alerta territorial si el riesgo predicho por el modelo para la próxima semana supera un umbral estadístico histórico, para visualizarlo posteriormente en el mapa.
- **Criterios de Aceptación (Gherkin):**

Gherkin

Feature: Generación de alertas por umbral

Scenario: Detección de incrementos anómalos de riesgo

Given las predicciones del modelo para el municipio "i" en la semana "t+1"

And el promedio histórico de casos para esa misma semana epidemiológica

When la predicción actual supera en un 30% (o más) la media histórica

Then el sistema inserta un registro en la tabla "alertas_territoriales" con nivel "ALERTA_NARANJA"

And el registro incluye el municipio, la fecha y la variable causal principal según la Feature Importance

And el umbral del 30% está respaldado por el análisis de percentiles de HU9

And el sistema genera tres niveles: ALERTA_VERDE (<15%), ALERTA_NARANJA (15-30%), ALERTA_ROJA (>30%)

EPICA 5 - Plataforma de Visualización Territorial (Web-GIS Dashboard)

- **Objetivo de la Épica:** Construir una interfaz de usuario interactiva y desacoplada que consuma las APIs del sistema para presentar mapas de riesgo (IPT), tendencias históricas y alertas preventivas de manera comprensible e intuitiva.
- **Métrica de Éxito:** Tiempo de carga del Dashboard (Page Load Time) inferior a 3 segundos, y correcta renderización del mapa coroplético (mapa de colores) sin errores de consola.

HU16: Desarrollo de API Geoespacial (Backend FastAPI)

- **Descripción:** Como desarrollador Frontend, necesito endpoints en la API que me entreguen los datos de riesgo e indicadores pre-procesados y unidos a la geometría ligera del municipio en formato GeoJSON, para no tener que hacer procesamientos pesados en el cliente.
- **Criterios de Aceptación (Gherkin):**

Gherkin

Feature: API REST de consumo de indicadores espaciales

Scenario: Petición del mapa de riesgo para una fecha específica

Given la base de datos PostGIS poblada con la tabla "fact_riesgo_territorial"

When el cliente envía una petición GET a "/api/v1/mapa/riesgo?fecha=2026-06-01"

Then la API realiza un JOIN espacial entre los indicadores y la capa "dim_municipios"

And aplica una función de simplificación espacial (ej. ST_Simplify en PostGIS)

And retorna un objeto FeatureCollection de GeoJSON con código HTTP 200 OK

And el payload de respuesta pesa menos de 5MB

And la función ST_Simplify se aplica con tolerancia 0.001 grados para preservar la legibilidad de los polígonos municipales

HU17: Renderizado del Mapa Coroplético Interactivo (Frontend)

- **Descripción:** Como analista de salud, necesito visualizar un mapa de Colombia coloreado según el nivel del Índice Preventivo Territorial (IPT) y las alertas de Machine Learning, para identificar rápidamente zonas de alta vulnerabilidad.
- **Criterios de Aceptación (Gherkin):**

Gherkin

Feature: Mapa de Riesgo Preventivo

Scenario: Visualización del IPT y Alertas mediante gradiente de color

Given el Dashboard principal cargado en el navegador

When el componente del mapa recibe el GeoJSON desde la API

Then los municipios se rellenan con una paleta de colores divergente (ej. verde=Bajo, amarillo=Medio, rojo=Alto) basada en el atributo "IPT_score"

And si un municipio tiene alerta activa, se superpone un ícono de advertencia (⚠) centrado en el centroide del municipio

And al hacer clic (hover) sobre un municipio, un tooltip muestra su nombre, casos actuales y contaminante principal

HU18: Panel de Control y Filtros Dinámicos (Frontend)

- **descripción:** Como usuario del sistema, necesito paneles de métricas generales (KPIs) y controles para filtrar la información por rango de tiempo y departamento, para realizar análisis focalizados.

Agregar el KPI de IPT promedio del Valle de Aburrá como widget adicional, que es la métrica resumen más importante del proyecto.

- **Criterios de Aceptación (Gherkin):**

Gherkin

Feature: Interactividad y Filtros del Dashboard

Scenario: Filtrado temporal y actualización reactiva

Given el Dashboard visualizando el mapa a nivel nacional

When el usuario ajusta el componente deslizante (slider) de la fecha

Then el Frontend envía una nueva petición a la API con los nuevos parámetros

And los widgets de texto (ej. "Total Casos Estimados", "Promedio PM2.5 Nacional") se actualizan

And el mapa redibuja sus colores para reflejar el estado de la nueva fecha seleccionada sin recargar la página completa

EPICA 6 - Validación (QA), Contenerización y Despliegue

- **Objetivo de la Épica:** Asegurar la calidad del código mediante pruebas automatizadas (CI), empaquetar los componentes del sistema en contenedores aislados, y desplegar el MVP en una infraestructura en la nube (ej. Railway/Render) garantizando su disponibilidad para la defensa académica.
- **Métrica de Éxito:** Pipeline de CI/CD en verde (pasando todas las pruebas), tiempo de despliegue automatizado < 10 minutos, y disponibilidad (Uptime) del 99% durante la semana de la presentación final.

HU19: Automatización de Pruebas de Integración (Backend)

- **Descripción:** Como ingeniero de QA, necesito una suite automatizada de pruebas de integración que valide los endpoints de la API construida en FastAPI, asegurando que retorne códigos de estado HTTP correctos y formatos GeoJSON válidos antes de cualquier despliegue.
- **Criterios de Aceptación (Gherkin):**

Gherkin

Feature: Pruebas de Integración de la API (CI)

Scenario: Validación de respuesta del endpoint geoespacial

Given el cliente de pruebas de FastAPI ("TestClient") configurado

And una base de datos de pruebas (test_db) con datos semilla (mock data)

When se ejecuta una petición GET a "/api/v1/mapa/riesgo"

Then la respuesta HTTP debe tener un status code 200

And el "Content-Type" debe ser "application/json"

And la estructura de la respuesta debe validar contra el esquema GeoJSON (FeatureCollection)

And el campo "type" de la respuesta es "FeatureCollection"

And cada feature tiene las propiedades "codigo_dane", "ipt_score" y "nivel_riesgo"

HU20: Contenerización del Sistema Multi-Capa (Docker Compose)

- **Descripción:** Como ingeniero DevOps, necesito crear los archivos Dockerfile para el Frontend y el Backend, y orquestarlos junto a la base de datos usando docker-compose, para garantizar una infraestructura inmutable y reproducible.
- **Criterios de Aceptación (Gherkin):**

Gherkin

Feature: Orquestación de Contenedores

Scenario: Despliegue unificado de la arquitectura

Given un "Dockerfile" optimizado para FastAPI (Backend) y otro para Streamlit (Frontend)

And el archivo "docker-compose.yml" vinculando los servicios web y la base de datos PostGIS

When el administrador ejecuta "docker-compose up --build" en un entorno limpio

Then los tres contenedores (db, api, web) se levantan e intercomunican en la misma red de Docker

And la política "restart: always" garantiza que si un proceso falla, el contenedor se reinicie automáticamente (minimizando el MTTR)

HU21: Despliegue en la Nube (Cloud Hosting)

- **Descripción:** Como arquitecto del sistema, necesito conectar nuestro repositorio de GitHub a un servicio de Cloud PaaS (como Render o Railway) para que cada cambio aprobado en la rama principal (main) se despliegue automáticamente en internet (Continuous Deployment).
- **Criterios de Aceptación:** *(Se documenta en formato lista de chequeo al ser infraestructura)*
 - El servicio de base de datos administrada PostgreSQL/PostGIS está provisionado en la nube.
 - Las variables de entorno (.env) seguras están inyectadas en la plataforma (credenciales, secretos).
 - El Web Service expone la aplicación Streamlit mediante una URL pública y segura (HTTPS).

HU22: Pruebas de Humo (Smoke Testing) en Producción

- **Descripción:** Como Product Owner, necesito realizar pruebas de humo una vez el sistema esté en producción para certificar que las funcionalidades críticas (visualizar el mapa, ver las alertas) operan correctamente bajo condiciones reales de red.
- **Criterios de Aceptación (Gherkin):**

Gherkin

Feature: Certificación de Entorno Productivo

Scenario: Navegación crítica exitosa (Happy Path)

Given el usuario accede a la URL pública de VitalRisk AI

When la página de inicio termina de cargar

Then el mapa de Colombia se renderiza sin errores en la consola del navegador

And el filtro de fechas interactúa correctamente con el backend en la nube

And el tiempo total de carga de la solicitud inicial no supera los 5 segundos

EPICA 7 - Presentación, Documentación y Defensa del MVP

- **Objetivo de la Épica:** Consolidar los artefactos técnicos, preparar un entorno de demostración infalible y estructurar la narrativa epidemiológica-tecnológica para la defensa final del proyecto en agosto de 2026.
- **Métrica de Éxito:** Repositorio calificado con un 100% en legibilidad técnica (README, diagramas) y ejecución exitosa de una demostración de 10 minutos sin errores de interrupción (Zero-downtime demo).

HU23: Documentación Técnica de Código Abierto (README & Swagger)

- **Descripción:** Como desarrollador, necesito un repositorio perfectamente documentado para que el jurado (o futuros colaboradores) puedan entender la arquitectura, reproducir el entorno y consumir la API sin asistencia directa.
- **Criterios de Aceptación (Checklist de Entregables):**
 - [] El archivo README.md contiene las instrucciones de instalación con Docker (docker-compose up).
 - [] Se incluye un diagrama de arquitectura (ej. estándar C4 o diagrama de bloques) exportado en imagen.
 - [] La API de FastAPI tiene la documentación automática habilitada (/docs - Swagger UI) con ejemplos claros de los parámetros (ej. fecha, municipio_id).
 - [] Las decisiones críticas (ej. por qué se usó XGBoost y no Deep Learning) están documentadas en la carpeta /docs/ADRs.

HU24: Construcción del Escenario "Golden Path" para la Demo

- **Descripción:** Como presentador, necesito un script en la base de datos que cargue un escenario histórico específico (por ejemplo, una semana crítica de contaminación e IRA en el Valle de Aburrá) para mostrar el funcionamiento del sistema sin depender de datos en tiempo real inestables.
- **Criterios de Aceptación (Checklist de Entregables):**
 - [] Se ha creado un script seed_demo_data.sql que inyecta datos estables y verificados en PostGIS.
 - [] El modelo XGBoost genera predicciones lógicas para este conjunto de datos semilla.

- [] El Dashboard (Streamlit) se inicializa por defecto en la fecha del escenario "Golden Path" para evitar tiempos de carga en blanco durante la presentación.

HU25: Elaboración del Pitch Deck (Narrativa Visual)

- **Descripción:** Como líder del proyecto, necesito una presentación de diapositivas (Pitch Deck) clara, ejecutiva y visual, que conecte el problema de salud pública con nuestra solución técnica.
- **Criterios de Aceptación (Checklist de Estructura - Máximo 10 Slides):**
 - [] **Problema:** Impacto territorial de las enfermedades respiratorias (Datos y contexto colombiano).
 - [] **Solución (VitalRisk AI):** Qué es y por qué es preventivo, no clínico.
 - [] **Arquitectura de Datos:** Diagrama del ETL (SIATA, INS, DANE -> PostGIS -> FastAPI).
 - [] **El Cerebro:** Explicación simple del IPT y la importancia de variables (XGBoost).
 - [] **Demo:** (Punto de transición al sistema en vivo).
 - [] **Trabajo Futuro:** Mención del módulo cardiovascular (Epic 4.5) como siguiente fase.

ejemplo(verificar con la grabacion de sustentacion)

La estructura de 10 slides debería ser: Problema → **Nuestros Datos** → Solución → Arquitectura → El Cerebro (IPT + ML) → Demo → Impacto → Trabajo Futuro

HU26: Simulacro de Defensa Técnica y "Murder Board"

- **Descripción:** Como equipo, necesitamos realizar sesiones de preguntas y respuestas difíciles (*Murder Boarding*) para anticipar las críticas del jurado sobre nuestra metodología y tecnología.
- **Criterios de Aceptación (Casos de prueba orales superados):**
 - [] El equipo puede explicar fluidamente cómo evitaron el "Data Leakage" en el entrenamiento.
 - [] El equipo puede justificar por qué el IPT está acotado de 0 a 100.

- ☐ El equipo puede explicar la diferencia entre causalidad y correlación en los resultados del modelo.
- ☐ El equipo reconoce claramente las limitaciones del MVP (ej. "La granularidad es municipal, no a nivel de barrio").
- ☐ El equipo puede explicar por qué eligieron RMSE y MAE como métricas y no AUC-ROC
- ☐ El equipo puede responder: "¿Por qué no usaron SARIMAX como SaluData de Bogotá?"